

Navigating the BLS API with R: A Step- by-Step Guide

Samer Hijjazi



What BLS Offers

The Bureau of Labor Statistics (BLS) offers a variety of public data on topics such as:

- Employment & Unemployment
- Job Openings & Labor Turnover
- Prices & Inflation
- Wages & Earnings

The Data Format

The BLS data is in a **time-series** format (primarily monthly data)

Year	Jan	Feb	Mar	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec
2015	5.7	5.5	5.4	5.4	5.6	5.3	5.2	5.1	5.0	5.0	5.1	5.0
2016	4.8	4.9	5.0	5.1	4.8	4.9	4.8	4.9	5.0	4.9	4.7	4.7
2017	4.7	4.6	4.4	4.4	4.4	4.3	4.3	4.4	4.3	4.2	4.2	4.1
2018	4.0	4.1	4.0	4.0	3.8	4.0	3.8	3.8	3.7	3.8	3.8	3.9
2019	4.0	3.8	3.8	3.7	3.6	3.6	3.7	3.6	3.5	3.6	3.6	3.6
2020	3.6	3.5	4.4	14.8	13.2	11.0	10.2	8.4	7.8	6.9	6.7	6.7
2021	6.4	6.2	6.1	6.1	5.8	5.9	5.4	5.1	4.7	4.5	4.2	3.9
2022	4.0	3.8	3.7	3.7	3.6	3.6	3.5	3.6	3.5	3.6	3.6	3.5
2023	3.5	3.6	3.5	3.4	3.6	3.6	3.5	3.7	3.8	3.9	3.7	3.8
2024	3.7	3.9	3.9	3.9	4.0	4.1	4.2	4.2	4.1	4.1	4.2	4.1
2025	4.0	4.1	4.2	4.2	4.2							

Average Unemployment Rate

Accessing the Data

The data is broken into data sets, each known as a **series**. Each series has a unique series ID which helps you identify which data to access.

A series ID is comprised of letters and numbers, with each digit/set of digits having a meaning.

Series ID Example

Here is an example of a series ID format

- CUUR0000SAF

Series ID Format Breakdown

CUUR0000SAF

- CU represents the prefix of the series ID. In this case, it represents the Consumer Price Index (CPI) for all urban consumers

Series ID Format Breakdown

CUUR0000SAF

- U means the data is not seasonally adjusted. S means the data is seasonally adjusted.

Series ID Format Breakdown

CUUR0000SAF

- R means the data is in a monthly format. S means the data is semi-annual

Series ID Format Breakdown

CUUR0000SAF

- 0000 represents the area code. This area code is for the average US city.

Series ID Format Breakdown

CUUR0000**SAF**

- SAF represents the item code. This item code is for food and beverages.

Series ID Format Breakdown

CUUR0000SAF

- Consumer Price Index for All Urban Consumers for food and beverages in the average US city, not seasonally adjusted (monthly)

To see a list of popular series, visit

<https://data.bls.gov/toppicks?survey=bls>

To see the structure of different Series IDs, along with a list of area codes, item codes, and more, visit

<https://www.bls.gov/help/hlpforma.htm>

Accessing the API (Version 2.0)

1. Request an API key from

<https://data.bls.gov/registrationEngine/>

- Provide your email address and organization (you can put 'self' for organization).

2. Access the API documentation from

https://www.bls.gov/developers/api_signature_v2.htm

Notes About the API

- You can request 500 queries per day, with 50 series IDs per query.
- Per query, you can request up to 20 years of data.
- You must renew the API key at least once every year.
- There are some limitations to data access.

More info can be found in their Q&A:

https://www.bls.gov/developers/api_faqs.htm

The R Package Needed



The Code

```
# load libraries
library(httr2)
library(tidyverse)

# Insert your API Key here
api_key

# API URL access point
url <-
"https://api.bls.gov/publicAPI/v2/timeseries/data/"
```

The Code

```
# Create the HTTP request
req <- request(url) |>

  # Use the POST method for the HTTP request
  req_method("POST") |>

  # parameters to pass through the HTTP request
  req_body_json(list(
    seriesid = list("CUUR0000SA0L1E", "CUUR0000SAM",
"CUUR0000SAF"),
    startyear = "2024",
    endyear = "2024",
    registrationKey = api_key
  )) |>

  # set the content type to application/json
  req_headers(
    `Content-Type` = "application/json"
  )
```


The Code

```
# Make the request to receive the response  
resp <- req_perform(req)  
  
# Parse the response. This returns a list  
data <- resp_body_json(resp)
```

The Code

```
# single series
df <- data$Results$series[[1]]$data |>
  map(~ as_tibble(.)) |>
  bind_rows() |>
  mutate(seriesId = data$Results$series[[1]]$seriesID)
```

df

A tibble: 12 × 6

	year	period	periodName	value	footnotes	seriesId
	<chr>	<chr>	<chr>	<chr>	<list>	<chr>
1	2024	M12	December	322.007	<named list [0]>	CUUR0000SA0L1E
2	2024	M11	November	321.947	<named list [0]>	CUUR0000SA0L1E
3	2024	M10	October	321.758	<named list [0]>	CUUR0000SA0L1E
4	2024	M09	September	321.109	<named list [0]>	CUUR0000SA0L1E
5	2024	M08	August	320.017	<named list [0]>	CUUR0000SA0L1E
6	2024	M07	July	319.214	<named list [0]>	CUUR0000SA0L1E
7	2024	M06	June	319.003	<named list [0]>	CUUR0000SA0L1E
8	2024	M05	May	318.629	<named list [0]>	CUUR0000SA0L1E
9	2024	M04	April	317.978	<named list [0]>	CUUR0000SA0L1E
10	2024	M03	March	317.088	<named list [0]>	CUUR0000SA0L1E
11	2024	M02	February	315.419	<named list [0]>	CUUR0000SA0L1E
12	2024	M01	January	313.623	<named list [0]>	CUUR0000SA0L1E

The Code

```
# multi series
series_list <- map(data$Results$series, function(series_item) {
  map(series_item$data, ~ as_tibble(.)) |>
    bind_rows() |>
    mutate(seriesId = series_item$seriesID)
})
```

```
df <- bind_rows(series_list)
```

```
df |> arrange(period)
```

```
# A tibble: 36 × 6
```

	year	period	periodName	value	footnotes	seriesId
	<chr>	<chr>	<chr>	<chr>	<list>	<chr>
1	2024	M01	January	313.623	<named list [0]>	CUUR0000SA0L1E
2	2024	M01	January	557.215	<named list [0]>	CUUR0000SAM
3	2024	M01	January	324.782	<named list [0]>	CUUR0000SAF
4	2024	M02	February	315.419	<named list [0]>	CUUR0000SA0L1E
5	2024	M02	February	557.236	<named list [0]>	CUUR0000SAM
6	2024	M02	February	325.190	<named list [0]>	CUUR0000SAF
7	2024	M03	March	317.088	<named list [0]>	CUUR0000SA0L1E
8	2024	M03	March	559.935	<named list [0]>	CUUR0000SAM
9	2024	M03	March	325.491	<named list [0]>	CUUR0000SAF
10	2024	M04	April	317.978	<named list [0]>	CUUR0000SA0L1E

```
# 26 more rows
```